

Transition Post-Quantique des Tunnels VPN

I) Transition Post-Quantique : Enjeux mathématiques et applicatifs

La sécurité des tunnels VPN actuels (RSA, ECC) repose sur des problèmes mathématiques classiques. RSA s'appuie sur la difficulté de factoriser le produit de deux grands nombres premiers, tandis qu'ECC repose sur le problème du logarithme discret sur les courbes elliptiques. Sur un ordinateur classique, casser une clé RSA à l'aide du meilleur algorithme connu (le crible algébrique, GNFS) nécessite un temps de calcul sous-exponentiel ce qui garantit la sécurité actuelle.

Cependant, un ordinateur quantique exécutant l'algorithme de Shor peut résoudre ces mêmes problèmes en temps polynomial, rendant RSA et ECC vulnérables. L'urgence d'abandonner ces standards réside dans la pratique du Store Now, Decrypt Later : les attaquants stockent dès aujourd'hui du trafic réseau chiffré dans l'attente de disposer de la puissance quantique nécessaire pour le déchiffrer.

Pour contrer cette menace, le NIST a standardisé la cryptographie post-quantique (PQC), en sélectionnant ML-KEM pour l'échange de clés et ML-DSA (Dilithium3) pour les signatures numériques. Ces algorithmes reposent sur la géométrie des réseaux euclidiens (Lattices), et plus spécifiquement sur le problème mathématique du Learning With Errors (LWE).

Le problème LWE (Learning With Errors)

Le principe du LWE est basé sur la difficulté de résoudre un système d'équations linéaires modulaires dès lors qu'on y introduit un petit bruit aléatoire. La relation mathématique fondamentale s'exprime ainsi :

$$A \cdot s + e \equiv b \pmod{q}$$

Dans cette équation :

- A est une grande matrice publique connue de tous.
- s est le vecteur secret (la clé privée).
- b est le vecteur public (la clé publique).
- e est un vecteur d'erreur gaussien généré aléatoirement (le bruit).

Sans le vecteur d'erreur e, retrouver le secret s serait trivial en utilisant une simple élimination de Gauss-Jordan. Mais avec l'ajout de e, le problème devient asymétrique et extrêmement complexe à inverser, y compris pour les algorithmes quantiques connus à ce jour.

Toutefois, ce changement de paradigme mathématique a un coût physique : les matrices utilisées génèrent des clés publiques et des signatures beaucoup plus volumineuses que les standards actuels. Pour évaluer ce surcoût (latence, fragmentation IP) sur le monde réel, ce projet s'appuie sur la bibliothèque liboqs du

projet Open Quantum Safe (OQS). Son intégration au sein d'un fork d'OpenVPN nous permettra de mesurer scientifiquement l'impact de ces nouvelles mathématiques sur les performances d'un réseau.

II) Méthodologie et Déploiement du Banc de Test

Afin d'obtenir des mesures de performances scientifiquement valables et reproductibles, l'évaluation des algorithmes post-quantiques nécessite un environnement de test rigoureusement contrôlé. L'infrastructure déployée repose sur une architecture virtualisée (Oracle VirtualBox) composée de trois machines virtuelles sous Ubuntu Server, réparties sur deux réseaux NAT. Cette séparation stricte oblige l'intégralité du trafic à transiter par un nœud central.

Concrètement :

- La première machine agit comme Client VPN sur le premier réseau (10.0.1.0/24), initiant les requêtes cryptographiques.
- La deuxième machine fait office de Routeur et émulateur WAN, interconnectant les deux réseaux.
- La troisième machine, le Serveur VPN, est isolée sur le second réseau (10.0.2.0/24) et héberge le service OpenVPN modifié intégrant la bibliothèque post-quantique.

Émulation des conditions réseau

Tester la cryptographie post-quantique sur un réseau virtuel parfait, sans latence ni perte, fausserait totalement l'analyse expérimentale. Les algorithmes PQC génèrent des clés et des signatures dont la taille excède largement la MTU standard (1500 octets), provoquant une inévitable fragmentation IP. Pour observer le comportement de ces paquets fragmentés dans des conditions réalistes, nous utilisons l'outil Traffic Control (`tc`) couplé au module `netem` sur le routeur central.

Commandes utilisées

@ Routeur :

Activation du routage IP :

```
# Dans /etc/sysctl.conf, décommenter:
net.ipv4.ip_forward=1
# Puis appliquer :
sudo sysctl -p
```

Mise en place de la latence artificielle :

```
sudo tc qdisc add dev enp0s3 root netem delay 50ms
```

(enp0s3 est l'interface côté réseau du serveur VPN)

@ Client :

Ajout d'une route vers le réseau du serveur via le routeur :

```
# Dans /etc/netplan/50-cloud-init.yaml, ajouter sous l'interface :
routes:
  - to: 10.0.2.0/24
    via: 10.0.1.4    # IP de l'interface enp0s8 du routeur
```

@ Serveur :

Ajout d'une route vers le réseau du client via le routeur :

```
# Dans /etc/netplan/50-cloud-init.yaml, ajouter sous l'interface :  
  routes:  
    - to: 10.0.1.0/24  
      via: 10.0.2.4    # IP de l'interface enp0s3 du routeur
```

III) Implémentation

Afin de mener à bien l'intégration des algorithmes post-quantiques, on utilise l'écosystème open-source OQS (Open Quantum Safe). L'implémentation a nécessité la compilation manuelle d'une chaîne cryptographique complète : la bibliothèque de base liboqs, le fournisseur cryptographique oqs-provider pour OpenSSL 3.0, et enfin un fork spécifique d'OpenVPN.

Pour la PKI (Infrastructure à Clés Publiques), nous avons abandonné les standards RSA/ECDSA au profit de l'algorithme de signature ML-DSA (Dilithium) pour générer l'autorité de certification (CA) ainsi que les certificats du serveur et du client. Concernant l'établissement du tunnel (handshake), les fichiers de configuration d'OpenVPN ont été adaptés pour évaluer deux approches :

- Approche Pure PQC : utilisation exclusive de ML-KEM pour l'échange de clés.
- Approche Hybride : combinaison d'une courbe elliptique classique (X25519) avec ML-KEM, garantissant une sécurité optimale même en cas de faille découverte dans les nouvelles mathématiques post-quantiques.

Compilation de la chaîne cryptographique

@ Serveur & @ Client — Compilation et installation de liboqs :

```
git clone https://github.com/open-quantum-safe/liboqs.git
cd liboqs
mkdir build && cd build
cmake -GNinja -DCMAKE_INSTALL_PREFIX=/usr/local ..
ninja
sudo ninja install
```

Génération de la PKI post-quantique

@ Serveur — Création de l'Autorité de Certification (CA) avec Dilithium3 :

```
openssl req -x509 -new -newkey dilithium3 \
-keyout ca.key -out ca.crt -nodes -days 365 \
-provider oqsprovider -provider default \
-subj "/CN=OQS-VPN-CA"
```

Génération du certificat Serveur signé par la CA :

```
openssl req -new -newkey dilithium3 \
-keyout server.key -out server.csr -nodes \
-provider oqsprovider -provider default \
-subj "/CN=VPN-Server"

openssl x509 -req -in server.csr -out server.crt \
-CA ca.crt -CAkey ca.key -CAcreateserial -days 365 \
-provider oqsprovider -provider default
```

Configuration d'OpenVPN

@ Serveur — Dans /etc/openvpn/server.conf :

Mode Pure PQC (Kyber768) :

```
tls-groups kyber768
```

Mode Hybride (X25519 + Kyber768) :

```
tls-groups x25519_kyber768
```

IV) Expérimentation

Ce projet repose sur la métrologie réseau et l'analyse comparative des trois scénarios cryptographiques configurés (Classique, Pure PQC, Hybride). L'objectif est de quantifier précisément le surcoût technique (overhead) induit par les réseaux euclidiens utilisés dans les algorithmes du NIST.

Métriques mesurées

- **Fragmentation IP** : à l'aide d'analyseurs de trames, le trafic est capturé lors du handshake pour mettre en évidence la fragmentation causée par le volume des clés publiques ML-KEM et des signatures ML-DSA (plusieurs kilo-octets) face à une MTU standard de 1500 octets.
- **Temps de handshake** : le temps global d'établissement du tunnel est chronométré et croisé avec les différentes latences artificielles injectées par le routeur.
- **Débit volumique (throughput)** : des tests de charge sont menés pour vérifier que le débit n'est pas impacté une fois le tunnel monté, le trafic encapsulé basculant sur un chiffrement symétrique standard (AES-GCM). Il convient de noter que la renégociation périodique des clés (rekeying) utilise également les algorithmes PQC et pourrait causer des micro-interruptions mesurables.
- **Ressources matérielles** : l'empreinte mémoire et la charge CPU sont relevées sur le serveur lors de connexions simultanées, afin d'évaluer la scalabilité de ces nouveaux standards.

Protocole de mesure

Afin de garantir la validité statistique des résultats, chaque scénario de test est répété un minimum de 30 fois. Les résultats sont présentés sous forme de moyenne arithmétique accompagnée de l'écart-type et de l'intervalle de confiance. Les valeurs aberrantes sont identifiées et, le cas échéant, exclues selon la méthode de l'IQR (InterQuartile Range).

Commandes utilisées

@ Serveur :

Capture du trafic réseau du handshake (port UDP 1194 par défaut d'OpenVPN) :

```
sudo tcpdump -i enp0s3 -w capture_pqc_handshake.pcap udp port 1194
```

Lancement de l'écouteur iperf3 pour la mesure de bande passante :

```
iperf3 -s
```

Monitoring en temps réel de la consommation CPU et RAM du processus OpenVPN (mesure toutes les secondes) :

```
pidstat -u -r -p $(pgrep openvpn) 1
```

@ Client :

Mesure du temps d'établissement de la connexion VPN (extraction du timestamp dans les logs locaux OpenVPN au moment où l'état passe à « Initialization Sequence Completed »).

Injection de trafic au travers du tunnel VPN monté (via l'interface tun0) pour mesurer le débit en TCP :

```
iperf3 -c 10.8.0.1 -t 30
```